



Основи тестування. Інструменти тестування Devops



Що таке тестування в DevOps?

DevOps — це прогресивний метод розробки та розгортання, який спрощує та оптимізує процеси між різними командами, дозволяючи розробникам, фахівцям з експлуатації, інженерам із забезпечення якості та аналітикам краще співпрацювати та впоратися зі стислими термінами.

Ще однією важливою особливістю сучасного забезпечення якості є автоматизація. Сьогодні типово створювати програмне забезпечення за короткі цикли спринту, і було б неможливо запустити велику кількість необхідних випадків керування вручну. У тестуванні якості DevOps автоматизація є обов'язковою умовою успішної роботи, оскільки вона зменшує кількість повторюваних завдань і скорочує час, необхідний для отримання результатів перевірки якості. Автоматизація є найбільшим засобом підтримки якості та швидкості.

Що таке тестування в DevOps?

Роль і навички тестувальників також були змінені. Сьогодні QA-тестери перетворюються на інженерів автоматизації, які володіють більшими технічними навичками, розуміють як функціональність програми, так і її технічне середовище, знають основи кодування та вміють його аналізувати. Вони знають, як використовувати засоби автоматизації та створювати сценарії паралельно з розробкою.

На відміну від раніших часів, тестувальник не є ізольованим членом, а скоріше важливою частиною команди, яка бере участь у процесі з самого початку. Інженери з автоматизації забезпечують якість протягом усього процесу та перевіряють правильне функціонування розробленої програми.

Переваги безперервного тестування

Існує чотири основні поточні процеси в культурі DevOps:

- Безперервна інтеграція
- Постійна доставка
- Постійне тестування
- Постійний моніторинг

Усі вони тісно переплетені. Безперервна доставка та інтеграція неможливі без контролю та моніторингу, оскільки вони потребують швидкої оцінки змін для частих високоякісних випусків.

Переваги безперервного тестування

Тестування програмного забезпечення для DevOps є невід'ємною частиною, яка має певні переваги, оскільки:

- Забезпечує швидкий і постійний зворотний зв'язок
- Виключає помилки на ранніх стадіях
- Підвищує ефективність процесу перевірки
- Прискорює швидкість змін
- Забезпечує бездоганну безперервність протягом усього SDLC (життєвого циклу розробки системи)
- Дозволяє розробникам швидше виводити нові функції на ринок
- Забезпечує безпеку системи
- Дозволяє менеджерам приймати кращі компромісні рішення

Оскільки DevOps руйнує бар'єри між Dev і Ops і заохочує кращу координацію між різними відділами, це гарантує, що кожен бере на себе відповідальність за якість. Таким чином, перевірка вбудована разом

Розробка стратегії тестування DevOps

Запровадження DevOps у вашій компанії безперечно матиме потужний вплив на стратегії контролю. Оскільки автоматизувати все неможливо, вам потрібна стратегія автоматизації, щоб отримати максимальну рентабельність інвестицій від витраченого часу та грошей. Добре спланована стратегія може збільшити швидкість розвитку та звільнити команди, щоб зосередитися на більш важливих завданнях. Ось так званий підручник з тестування DevOps, який допоможе вам розробити правильну стратегію автоматизації.

Розробка стратегії тестування DevOps

Виберіть правильні випадки для автоматизації

Які випадки ви обираєте для автоматизації: повторювані тести, випадки високого ризику, великі набори даних чи перевірки для різних браузерів і середовищ? Це залежить від сервісу, який ви розробляєте, і від можливостей вашої команди. Ваше завдання полягає в тому, щоб автоматизувати випадки, які принесуть найбільшу користь для процесу розробки та всієї організації.

Впроваджуйте автоматизацію протягом усього спринту

Швидкі випуски можуть бути досягнуті, лише якщо розробка та перевірка завершуються одночасно в кінці спринту. Тому перевірку якості слід починати якомога раніше. Тести на основі ключових слів можна писати до розробки нових функціональних можливостей, оскільки вони співвідносяться із завданням, але не вдаються в деталі. Під час процесу автоматизоване модульне тестування виконує дії, пише сценарії, щойно код стає доступним, і виконується безперервно.

Розробка стратегії тестування DevOps

Продовжуйте застосовувати автоматизовані кейси

Створені кейси через деякий час можуть стати складними та неактуальними, тому вам слід розглянути можливість створення гнучких тестів. Найкраще рішення — писати невеликі кейси, не перевантажуючи їх десятками кроків, починаючи зверху. Натомість розділіть програми на менші кроки та перевірте їх окремо.

Другий спосіб створити автоматичні тести, які триватимуть довго, — зробити їх незалежними від інтерфейсу користувача. Не включайте назви елементів інтерфейсу в сценарій, оскільки вони можуть змінитися під час розробки. Замість цього використовуйте функції серверної частини, щоб підтримувати актуальність тестових випадків і захищати їх від неминучих змін.

Стратегічне планування тестування: етапи та методології

Аналіз вимог: Зрозуміти обсяг проекту, його цілі та вимоги користувачів для визначення цілей і пріоритетів тестування.

Розробка тестової стратегії: Визначити загальний підхід до тестування, включаючи типи тестів, ресурси, строки виконання та стратегії управління ризиками.

Проектування тест-кейсів: Розробити комплексні тест-кейси на основі вимог, користувацьких історій та тестових сценаріїв для забезпечення достатнього покриття.

Підготовка тестового середовища: Підготувати необхідну інфраструктуру, інструменти та ресурси для виконання тестів у різних середовищах.

Виконання тестування: Систематично виконувати тест-кейси, фіксувати результати та контролювати наявність дефектів, забезпечуючи повне покриття функціональних та нефункціональних аспектів.

Управління дефектами: Відстежувати та пріоритезувати дефекти, співпрацювати з розробниками для їх усунення та перевіряти виправлення шляхом повторного тестування.

Закриття тестування: Оцінити результати тестування, задокументувати отримані уроки та підготувати звіти про тестування для зацікавлених сторін.

Створення ефективних тест-кейсів

Чіткі цілі: Визначте чіткі цілі для кожного тест-кейсу, включаючи конкретну функціональність або сценарій, що тестується.

Структура тест-кейсу: Логічно структуруйте тест-кейси, з чітким описом **вхідних даних, очікуваних результатів та передумов.**

Граничні умови: Включайте тест-кейси, які перевіряють граничні умови, крайові випадки та некоректні вхідні дані для виявлення потенційних проблем.

Позитивне та негативне тестування: Розробляйте тест-кейси для перевірки як очікуваної (позитивної), так і неочікуваної (негативної) поведінки системи.

Варіація даних: Тестуйте з різними вхідними даними, щоб забезпечити **стійкість та надійність** у різних сценаріях.

Повторне використання: Створюйте **тест-кейси, що можна повторно використовувати**, для підвищення ефективності та підтримуваності процесу тестування.

Відстежуваність: Забезпечте, щоб кожен тест-кейс **відслідковувався до конкретних вимог або користувацьких історій**, щоб демонструвати покриття.

Пріоритизація: Пріоритезуйте тест-кейси на основі **ризиків, критичності та бізнес-значущості**, щоб ефективно зосередити зусилля тестування.

Життєвий цикл багу: звітування та відстеження дефектів

Виявлення багів: Виявляйте та документуйте дефекти, які виникають під час тестування або за відгуками користувачів.

Звітування багів: Повідомляйте про дефекти у **стандартизованому форматі**, включаючи детальний опис, кроки для відтворення та відповідні скріншоти або логи.

Пріоритизація багів: Пріоритезуйте дефекти на основі **серйозності, впливу на функціональність та бізнес-пріоритетів**.

Призначення багів: Призначайте дефекти відповідним членам команди, таким як **розробники або QA-інженери**, для їх усунення.

Усунення багів: Розробники аналізують, відтворюють і усувають дефекти, відповідно оновлюючи статус багу.

Перевірка багів: QA-інженери перевіряють виправлення багів, щоб переконатися, що вони ефективно усувають повідомлену проблему.

Закриття багів: Після перевірки дефекти закриваються, а відповідна документація оновлюється.

Аналіз багів: Періодично аналізуйте дані про дефекти для **виявлення тенденцій, причин виникнення та сфер для покращення** процесу розробки.

Автоматизація тестування: переваги та виклики

Автоматизація тестування оптимізує процес тестування, забезпечуючи такі переваги, як **підвищена ефективність, покращена точність та розширене покриття тестів**.

Автоматизовані тести дозволяють **виявляти дефекти на ранніх етапах**, що з часом призводить до економії коштів.

Водночас впровадження автоматизації потребує **початкових інвестицій та постійного обслуговування**.

Вибір **підходящих тестів для автоматизації, підбір відповідних інструментів та управління помилковими позитивними/негативними результатами** є викликами.

Незважаючи на ці труднощі, **ефективна автоматизація тестування** може значно покращити **якість програмного забезпечення та ефективність розробки**.

Вступ до інструментів автоматизації тестування

Інструменти автоматизації тестування полегшують створення, виконання та управління автоматизованими тестами.

Ці інструменти мають різноманітні **функції та можливості**, щоб підтримувати різні потреби тестування.

Типові функції включають: **створення тестових скриптів, управління тест-кейсами, звітність та інтеграцію з процесами розробки і CI/CD.**

Інструменти автоматизації тестування варіюються від **відкритих фреймворків**, таких як Selenium та Appium, до **комерційних рішень**, таких як HP UFT та Tricentis Tosca.

Кожен інструмент має свої **переваги та придатність для конкретних сценаріїв тестування, платформ і технологій.**

Використовуючи інструменти автоматизації тестування, організації можуть **оптимізувати процеси тестування, підвищити ефективність та надійно поставляти високоякісні програмні продукти.**





Типи тестування

Основи модульного тестування

Модульне тестування перевіряє функціональність окремих **модулів або компонентів** програмного забезпечення в ізоляції.

Воно забезпечує, що кожен модуль **поводиться відповідно до свого проєктного призначення**.

Створюючи та виконуючи **тест-кейси для цих модулів незалежно**, розробники можуть **виявляти помилки на ранніх етапах** та оптимізувати процеси відладки і підтримки.

Модульне тестування є невід'ємною частиною **DevOps-практик**, підвищуючи **якість програмного забезпечення** та забезпечуючи **безперебійну інтеграцію і доставку**.



Дослідження інтеграційного тестування

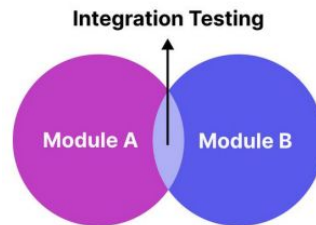
Інтеграційне тестування оцінює взаємодію між різними модулями або компонентами програмного забезпечення.

Воно забезпечує, що ці модулі **працюють узгоджено та правильно як єдина система**.

Перевіряючи **інтерфейси та взаємодії між модулями**, інтеграційне тестування виявляє проблеми, що виникають у процесі їх інтеграції.

Цей тип тестування є критично важливим для **підтвердження поведінки системи в цілому** та забезпечення **безшовної роботи всіх компонентів**.

Інтеграційне тестування є невід'ємною частиною **життєвого циклу розробки програмного забезпечення** та відіграє важливу роль у досягненні **надійних та стійких програмних продуктів**.



Огляд системного тестування



Системне тестування оцінює **повну та інтегровану програмну систему**, щоб переконатися, що вона відповідає визначеним вимогам і правильно функціонує в призначеному середовищі.

На відміну від **модульного та інтеграційного тестування**, які зосереджені на окремих компонентах або їх взаємодії, системне тестування **оцінює поведінку всієї системи**.

Воно перевіряє різні аспекти, такі як **функціональність, продуктивність, безпека та надійність**.

Системне тестування є критично важливим для **виявлення дефектів**, що можуть виникати через взаємодію між різними компонентами або підсистемами.

Проведення **всебічного системного тестування** дозволяє організаціям переконатися, що їхнє програмне забезпечення **відповідає очікуванням користувачів і надає заплановану цінність**.

Розуміння приймального тестування

Приймальне тестування, також відоме як **тестування приймання користувачем (UAT)**, оцінює, чи відповідає програмна система **бізнес-вимогам** і чи придатна для передачі кінцевим користувачам.



Воно включає перевірку **функціональності, зручності використання та загальної продуктивності системи** відповідно до заздалегідь визначених критеріїв або користувацьких історій.

Приймальне тестування зазвичай проводиться **на завершальному етапі життєвого циклу розробки програмного забезпечення** і виконується зацікавленими сторонами або безпосередньо кінцевими користувачами.

Основна мета приймального тестування — переконатися, що програмне забезпечення **виконує своє призначення та ефективно задовольняє потреби користувачів**.

Успішне проходження приймального тестування свідчить про **готовність до впровадження та прийняття користувачами**, підтверджуючи відповідність програмного забезпечення бізнес-цілям.



Рівні тестування

Деталі компонентного тестування

Компонентне тестування, також відоме як **модульне тестування** або **юніт-тестування**, зосереджується на перевірці **окремих модулів або компонентів** програмної системи.

Воно включає тестування **кожного модуля в ізоляції**, щоб переконатися, що він працює відповідно до **своїх проектних специфікацій**.

Компонентне тестування зазвичай проводять **розробники на ранніх етапах процесу розробки**.

Ретельне тестування кожного компонента дозволяє **виявляти та виправляти дефекти на ранніх етапах**, що сприяє загальній якості програмного забезпечення та зменшує ризик проблем під час інтеграційного та системного тестування.

Компонентне тестування є **необхідною практикою** для забезпечення **надійності та стійкості компонентів програмного забезпечення** перед їх інтеграцією в більші системи.

Дослідження інтерфейсного тестування

Інтерфейсне тестування зосереджується на перевірці **взаємодії та комунікації між різними компонентами або системами програмного забезпечення**.

Воно забезпечує, що інтерфейси, такі як **API, веб-сервіси та користувацькі інтерфейси**, працюють правильно та обмінюються даними відповідно до очікувань.

Інтерфейсне тестування перевіряє **сумісність, надійність та цілісність інтерфейсів**, щоб забезпечити **безшовну інтеграцію між різними модулями програмного забезпечення або зовнішніми системами**.

Цей тип тестування допомагає **виявляти будь-які непослідовності або помилки у обміні даними**, забезпечуючи ефективну взаємодію компонентів між собою.

Інтерфейсне тестування є критично важливим для **підтвердження сумісності програмних систем та забезпечення стабільної роботи у реальних умовах**.

Комплексне сквозне (End-to-End) тестування

Сквозне тестування оцінює **функціональність та продуктивність всієї програмної системи** у змодельованому реальному середовищі.

Воно включає тестування системи **від початку до кінця**, охоплюючи всі інтегровані компоненти, підсистеми та зовнішні залежності.

Мета сквозного тестування — **підтвердити поведінку та робочий процес системи**, переконатися, що вона відповідає **вимогам користувачів та бізнес-цілям**.

Симулюючи типові сценарії користувача, сквозне тестування допомагає **виявляти потенційні проблеми або вузькі місця**, що можуть виникнути під час реального використання.

Цей рівень тестування надає **цінну інформацію про надійність, масштабованість та зручність використання системи**, забезпечуючи **безперебійну взаємодію користувача з програмою**.



Техніки тестування

Розкриття чорного ящика (Black Box Testing)

Тестування чорного ящика, також відоме як **поведінкове тестування** або **функціональне тестування**, оцінює **функціональність програмного забезпечення**, не враховуючи його внутрішню структуру коду.

Тестувальники аналізують **вхідні та вихідні дані системи**, а також її **зовнішню поведінку**, щоб перевірити відповідність **встановленим вимогам**.

Ця техніка розглядає програмне забезпечення як **чорний ящик**, зосереджуючись виключно на **спостережуваній поведінці та функціональності** з точки зору користувача.

Тестування чорного ящика дозволяє виявляти **дефекти, помилки та непослідовності** у поведінці програмного забезпечення без необхідності доступу до **внутрішніх деталей реалізації**.

Симулюючи різні сценарії користувачів, тестування чорного ящика допомагає переконатися, що програмне забезпечення **відповідає очікуванням користувачів та працює належним чином у різних умовах використання**.

Розкриття білого ящика (White Box Testing)

Тестування білого ящика, також відоме як **структурне тестування** або **тестування скляного ящика**, аналізує **внутрішню структуру та реалізацію коду** програмного забезпечення.

Тестувальники вивчають **внутрішню логіку, шляхи виконання та структуру коду**, щоб розробити тест-кейси, які забезпечують **повне покриття коду**.

На відміну від тестування чорного ящика, яке зосереджене на зовнішній поведінці, **тестування білого ящика досліджує внутрішню роботу програмного забезпечення**, включно з **потокм керування, потоком даних та умовними операторами**.

Ця техніка дозволяє виявляти **помилки, дефекти та вразливості**, що приховані у кодовій базі.

Оцінюючи **цілісність та правильність коду**, тестування білого ящика допомагає **покращити якість коду, виявити потенційні проблеми безпеки та підвищити загальну надійність програмного забезпечення**.

Розкриття сірого ящика (Grey Box Testing)

Тестування сірого ящика поєднує елементи **тестування чорного та білого ящика**, забезпечуючи збалансований підхід до тестування програмного забезпечення.

Тестувальники мають **часткове знання внутрішньої роботи програмного забезпечення**, водночас оцінюючи його **зовнішню поведінку**.

Ця техніка дозволяє розробляти **тест-кейси на основі поєднання сценаріїв користувача та знань про внутрішню структуру коду**.

Тестування сірого ящика допомагає виявляти **дефекти та вразливості**, які можуть виникати як через зовнішні взаємодії, так і через **внутрішню логіку коду**.

Використовуючи цей **гібридний підхід**, тестування сірого ящика забезпечує **комплексну оцінку функціональності, продуктивності та безпеки програмного забезпечення**, сприяючи **підвищенню якості та надійності ПЗ**.



Інструменти тестування Devops

QuerySurge



QuerySurge — це розумне рішення для тестування даних перше у своєму роді повне рішення DevOps для безперервного тестування даних.

Ключові характеристики

- Надійний API із понад 60 викликами
- Легко інтегрується в конвеєр DevOps для безперервного тестування
- Швидко перевіряє великі обсяги даних
- Перевіряє складні правила перетворення між кількома вихідними та цільовими системами
- Виявляє вимоги та зміни коду, відповідно оновлює тести та сповіщає членів команди про ці зміни
- Надає детальну аналітику та аналітику даних

Jenkins

Jenkins — це інструмент тестування DevOps з відкритим кодом. Його можна використовувати для автоматизації всіх типів завдань, таких як створення, тестування та розгортання програмного забезпечення. Цей інструмент дозволяє розробникам швидко знаходити та вирішувати дефекти у своїй базі коду, щоб автоматизувати тестування своїх збірок.



Jenkins

Ключові характеристики

- Надає підтримку для масштабування до великої кількості вузлів і рівномірного розподілу робочого навантаження
- Сумісний з усіма ОС і версіями Linux, Mac OS або Windows
- Він пропонує легку інсталяцію, оскільки Jenkins постачається як файл WAR. Усе, що потрібно зробити користувачеві, — це закинути WAR у ваш контейнер JEE, і ваша установка буде готова до запуску.
- Jenkins можна легко встановити та налаштувати за допомогою веб-інтерфейсу
- Він може легко розподілити роботу між кількома машинами

Bamboo

Bamboo — це інструмент постійної інтеграції, який також використовується для тестування DevOps. Він виконує автоматичне збирання, тестування та випуски в одному місці. Він підтримує багато технологій і мов, як-от відра Ducker, Git, SVN, Mercurial і Amazon S3.



Ключові характеристики

- Налаштувати Bamboo досить просто
- Виконує тести паралельних партій
- Функція дозволів для кожного середовища дозволяє розробникам і спеціалістам із забезпечення якості розгортати їх у своїх середовищах
- Він може запускати збірки на основі змін, виявлених у сховищі, push-повідомлень від Bitbucket

Jmeter

Apache JMeter — це інструмент навантажувального тестування з відкритим кодом. Він призначений для вимірювання продуктивності веб-сайтів. Цей інструмент навантажувального тестування можна використовувати в методології DevOps.



Ключові характеристики

- JMeter дозволяє виконувати тест навантаження та продуктивності для різних типів серверів.
- Цей інструмент навантажувального тестування зберігає свої плани тестування у форматі XML, що дозволяє користувачам створювати план тестування за допомогою текстового редактора.
- Інструмент можна використовувати для автоматизованого та функціонального тестування додатків.

Selenium

Selenium є найпопулярнішим інструментом автоматизованого тестування. Його спеціально розроблено для підтримки автоматизованого тестування широкого діапазону браузерів.



Ключові характеристики

- Підтримка паралельного виконання тесту скорочує час, витрачений на виконання тесту
- Selenium потребує набагато менше ресурсів порівняно з іншими інструментами тестування.
- Тестові приклади, підготовлені за допомогою цього інструменту тестування, можна виконувати на будь-якій ОС
- Він підтримує такі популярні мови програмування, як Java, Python, C#, Perl, PHP і JavaScript.

Appium

Appium — це інструмент автоматизації з відкритим кодом для мобільних додатків. Це дозволяє користувачам тестувати всі види рідних, мобільних, веб- та гібридних програм. Він також підтримує автоматизовані тести на емуляторах і симуляторах. Він активно використовується в процесі DevOps.



Ключові характеристики

- Легкий процес налаштування.
- Це проста програма, яка вимагає дуже мало пам'яті для процесу тестування.
- Тестування нативних програм не потребує SDK. Він пропонує стандартні API автоматизації, які можна використовувати на всіх типах платформ.

Soap UI



SoapUI — це міжплатформний безкоштовний інструмент для тестування API з відкритим кодом для SOAP і REST. Широко використовується в інструменті тестування DevOps для виконання функціонального та навантажувального тестування API.

Ключові характеристики

- Графічний інтерфейс програмного забезпечення простий у обробці та використанні
- Функція перевірки вразливостей допомагає захистити веб-сайт від хакерів і вірусів.
- Можна зробити детальний аналіз за допомогою функції звітності.
- Функція SQL Injection надає деякі стандартні запити SQL і методи для визначення слабких місць програми.

CruiseControl



CruiseControl — це інструмент безперервної інтеграції. Його веб-інтерфейс надає деталі поточної та попередніх збірок. Він також забезпечує підтримку віддаленого керування.

Ключові характеристики

- Інтеграція з різними системами керування джерелами, такими як csv, svn, git, hg, perforce, clearcase, файлова система тощо.
- Це дозволяє створювати кілька проектів на одному сервері
- Це забезпечує бездоганну інтеграцію з іншими зовнішніми інструментами, такими як NAnt, NDepend, NUnit, MBUnit і Visual Studio

Vagrant



Vagrant — це інструмент тестування DevOps. Він використовується для створення та керування середовищами віртуальних машин в одному робочому процесі. Він пропонує простий у використанні робочий процес і фокусується на автоматизації. Це також скорочує час налаштування середовища розробки та підвищує парність виробництва.

Ключові характеристики

- Простий, легкий у використанні та потужний інструмент
- Vagrant інтегрується з існуючими інструментами керування конфігурацією, такими як Chef, Puppet, Ansible або Salt
- Vagrant бездоганно працює в ОС Mac, Linux і Windows
- Ніякого складного процесу налаштування, лише завантажте та встановіть
- Створіть єдиний файл для проектів для опису типу машини та програмного забезпечення, яке користувачі хочуть встановити

PagerDuty

PagerDuty — це інструмент DevOps, який допомагає компаніям підвищити репутацію свого бренду. Це рішення для керування інцидентами, яке підтримує стратегію CI. Цей інструмент тестування DevOps допомагає командам створювати високопродуктивні програми.

Ключові характеристики

- Надає сповіщення в реальному часі
- Надійний і багатий засіб оповіщення
- Групування та збагачення подій
- Отримайте видимість критичних систем і програм
- Легко виявляйте та вирішуйте інциденти від розробки до виробництва
- Він пропонує систему співпраці в реальному часі та звітування користувачів
- Він підтримує розширюваність платформи
- Це дозволяє планувати та автоматизувати ескалацію
- Повна видимість у середовищах розробки та виробництва
- Аналіз подій для ефективної інформації



Snort



Snort — це потужний безкоштовний інструмент із відкритим вихідним кодом, який допомагає виявляти зловмисників. Він також висвітлює зловмисні атаки на систему. Це дозволяє аналізувати трафік у реальному часі та реєструвати пакети.

Ключові характеристики

- Виконує аналіз протоколу та пошук вмісту
- Він дозволяє виявляти атаки на основі сигнатур шляхом аналізу пакетів
- Він пропонує аналіз трафіку в реальному часі, виявлення протоколювання пакетів, переповнення буфера тощо.

Docker



Docker — це пакет технологій DevOps. Це дозволяє командам DevOps створювати, надсилати та запускати розподілені програми. Цей інструмент дозволяє користувачам збирати програми з компонентів і працювати разом. Ця платформа з відкритим вихідним кодом для керування контейнерами програми як однієї групи.

Ключові характеристики

- Платформа CaaS Ready працює з вбудованою оркестровкою
- Гнучке керування зображеннями за допомогою приватного реєстру для зберігання та керування зображеннями та налаштування кешу зображень
- Ізолює програми в контейнерах, щоб усунути конфлікти для підвищення безпеки

Puppet Enterprise



Інструмент Puppet Enterprise усуває ручну роботу для процесу доставки програмного забезпечення. Це допомагає розробнику швидко постачати чудове програмне забезпечення

Ключові характеристики

- Інструмент Puppet Enterprise усуває ручну роботу для процесу доставки програмного забезпечення. Це допомагає розробнику швидко постачати чудове програмне забезпечення
- Моделюйте та керуйте своїм середовищем
- Інтелектуальна оркестровка та візуальні робочі процеси
- Звітування з урахуванням контексту в реальному часі
- Визначає і постійно підтримує інфраструктуру
- Він перевіряє та звітує про пакети, що працюють через інфраструктуру
- Виявлення та усунення конфлікту бажаного стану

UpGuard



UpGuard аналізує дані про мережі до пристрою в одному числовому значенні. Це дає швидке відчуття ризику безпеки.

Ключові характеристики

- UpGuard допомагає компаніям у всьому світі отримати доступ до своїх технологій
- Цей інструмент DevOps дозволяє збільшити швидкість доставки програмного забезпечення. Це досягається шляхом автоматизації ряду процесів і технологій.
- Це дозволяє користувачам довіряти конфіденційні дані третім особам
- Процедури, які використовуються для керування активами, такі ж важливі, як і самі конфігурації

UpGuard



AppVerify — це рішення для безперервного тестування, розроблене для бізнес-додатків. AppVerify дозволяє тестувати весь бізнес-процес, включаючи всі програми сторонніх розробників, без додавання плагінів або написання жодного рядка коду. Це полегшує безперервне тестування для команд Agile та DevOps.

Ключові характеристики

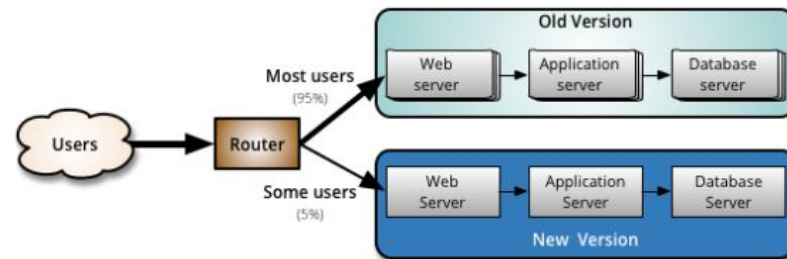
- Тестуйте будь-яку програму, відтворюючи однаковий досвід користувача з усіх ваших точок доступу: тонких, жирних клієнтів і веб-порталів
- AppVerify відтворює взаємодію користувачів із вашою програмою та надає цінні показники взаємодії кінцевих користувачів, включаючи знімки екрана збоїв.
- Сценарії створюються автоматично, коли ви використовуєте програму, і їх можна легко редагувати без програмування.
- Потім їх можна легко застосувати до доступних модулів тестування продуктивності та моніторингу програм.



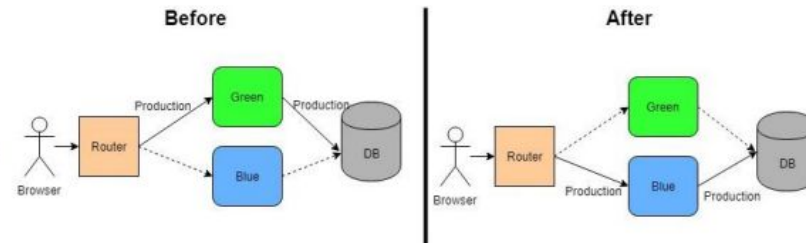
Просунутий

Тестування у продакшені

Canary Releases (Канарейкові релізи): При канарейковому релізі **невелика частина користувачів або трафіку** направляється на нову версію програмного забезпечення, тоді як більшість продовжує користуватися стабільною версією. Це дозволяє командам **моніторити продуктивність та збирати відгуки** перед поступовим розгортанням оновлення для всіх користувачів.



Blue-Green Deployments (Сині-зелені розгортання): Сині-зелені розгортання передбачають **підтримку двох ідентичних продакшн-середовищ**: одне активне (синє) і одне неактивне (зелене). **Нова версія ПЗ розгортається у неактивному середовищі** та проходить ретельне тестування. Після підтвердження її працездатності, **трафік переключається зі старого середовища на нове**, що мінімізує час простою та ризики.



Штучний інтелект та машинне навчання у тестуванні: поточні застосування та перспективи



Штучний інтелект (ШІ) та машинне навчання все активніше інтегруються у процеси тестування для **підвищення ефективності та результативності**.

Поточні застосування включають: **генерацію тест-кейсів, предиктивну аналітику для виявлення дефектів, автоматизоване обслуговування тестів та інтелектуальну оптимізацію тестування**.

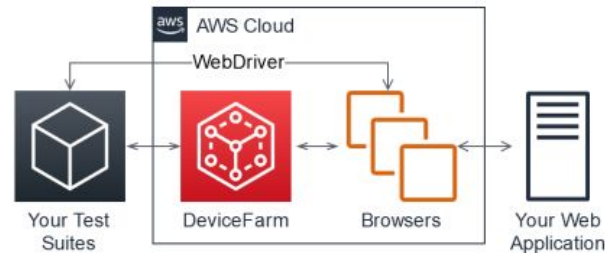
Ці технології забезпечують **швидше виконання тестів, підвищену точність у виявленні дефектів та кращу ефективність використання ресурсів тестування**.

У майбутньому очікується, що ШІ та машинне навчання **ще більше революціонізують тестування**, забезпечуючи **автономне тестування, самовідновлювані тестові набори та розширене виявлення аномалій**.

Впроваджуючи ШІ та машинне навчання у тестування, організації можуть **залишатися на передовій, постачати високоякісне програмне забезпечення та ефективно задовольняти змінні вимоги користувачів**.



Віртуалізація та тестування у хмарі: інструменти та платформи



Віртуалізація та хмарне тестування використовують ресурси хмарних обчислень для **ефективного та економного виконання тестувальних завдань**.

Платформи, такі як **Amazon Web Services (AWS)** та **Microsoft Azure**, пропонують різноманітні сервіси та інструменти, спеціально призначені для тестування.

У AWS тестувальники можуть використовувати сервіси, такі як **Amazon EC2** для масштабованих віртуалізованих тестових середовищ, **AWS Device Farm** для тестування мобільних додатків та **AWS Lambda** для виконання серверлес-тестів.

Подібно, Azure надає сервіси, такі як **Azure Virtual Machines** для гнучких тестових середовищ, **Azure DevTest Labs** для управління тестовими середовищами та **Azure App Service** для тестування веб-додатків.

Використовуючи ці платформи, організації можуть досягти **більшої гнучкості, масштабованості та економії коштів** у процесі тестування, що в кінцевому результаті призводить до **покращення якості програмного забезпечення та скорочення часу виходу на ринок**.



DevSecOps: Інтеграція безпеки у DevOps

sonarqube



DevSecOps підкреслює необхідність інтеграції практик безпеки на кожному етапі DevOps-пайплайну, включно з тестуванням. У цьому контексті тестування відіграє ключову роль у ранньому виявленні та усуненні вразливостей безпеки впродовж життєвого циклу розробки програмного забезпечення. Завдяки впровадженню тестування безпеки в автоматизовані CI/CD-пайплайни, DevSecOps забезпечує послідовне та ефективне застосування заходів безпеки протягом всього процесу розробки. Цей підхід включає:

- **Автоматизоване тестування безпеки:** інтеграція інструментів та методів автоматизованого тестування безпеки, таких як статичне тестування безпеки додатків (SAST), динамічне тестування безпеки додатків (DAST) та інтерактивне тестування безпеки додатків (IAST), у CI/CD-пайплайн для виявлення вразливостей у кодовій базі.
- **Перевірка коду на безпеку:** проведення регулярних перевірок коду з акцентом на безпеку в рамках процесу рев'ю коду для виявлення та усунення вразливостей у коді додатку.
- **Сканування на вразливості:** регулярне сканування залежностей, бібліотек та сторонніх компонентів, що використовуються у додатку, для виявлення та усунення відомих вразливостей безпеки.
- **Тестування на проникнення (Penetration Testing):** проведення тестів на проникнення для моделювання реальних атак та виявлення потенційних слабких місць у інфраструктурі додатку та його конфігураціях.

Нові тенденції у тестуванні DevOps

- **Shift-Right Testing (тестування «праворуч»):** акцент на тестуванні в продуктивних середовищах для отримання реальних відгуків та даних, що доповнює традиційні підходи shift-left.
- **Chaos Engineering (інженерія хаосу):** проведення контрольованих експериментів для виявлення слабких місць та можливих режимів відмов у розподілених системах, що підвищує їхню стійкість і надійність.
- **AI-driven Testing (тестування за допомогою ШІ):** використання штучного інтелекту та машинного навчання для автоматизації генерації тестів, їхнього виконання та аналізу, підвищуючи ефективність і якість тестування.
- **Тестування контейнеризації та мікросервісів:** розробка стратегій і застосування інструментів тестування, спеціально орієнтованих на контейнеризовані та мікросервісні архітектури, для забезпечення сумісності, масштабованості та продуктивності.
- **Compliance as Code (відповідність як код):** впровадження перевірок відповідності та контрольних механізмів як коду для автоматизації та забезпечення дотримання нормативних вимог у всьому DevOps-пайплайні.

Питання та відповіді